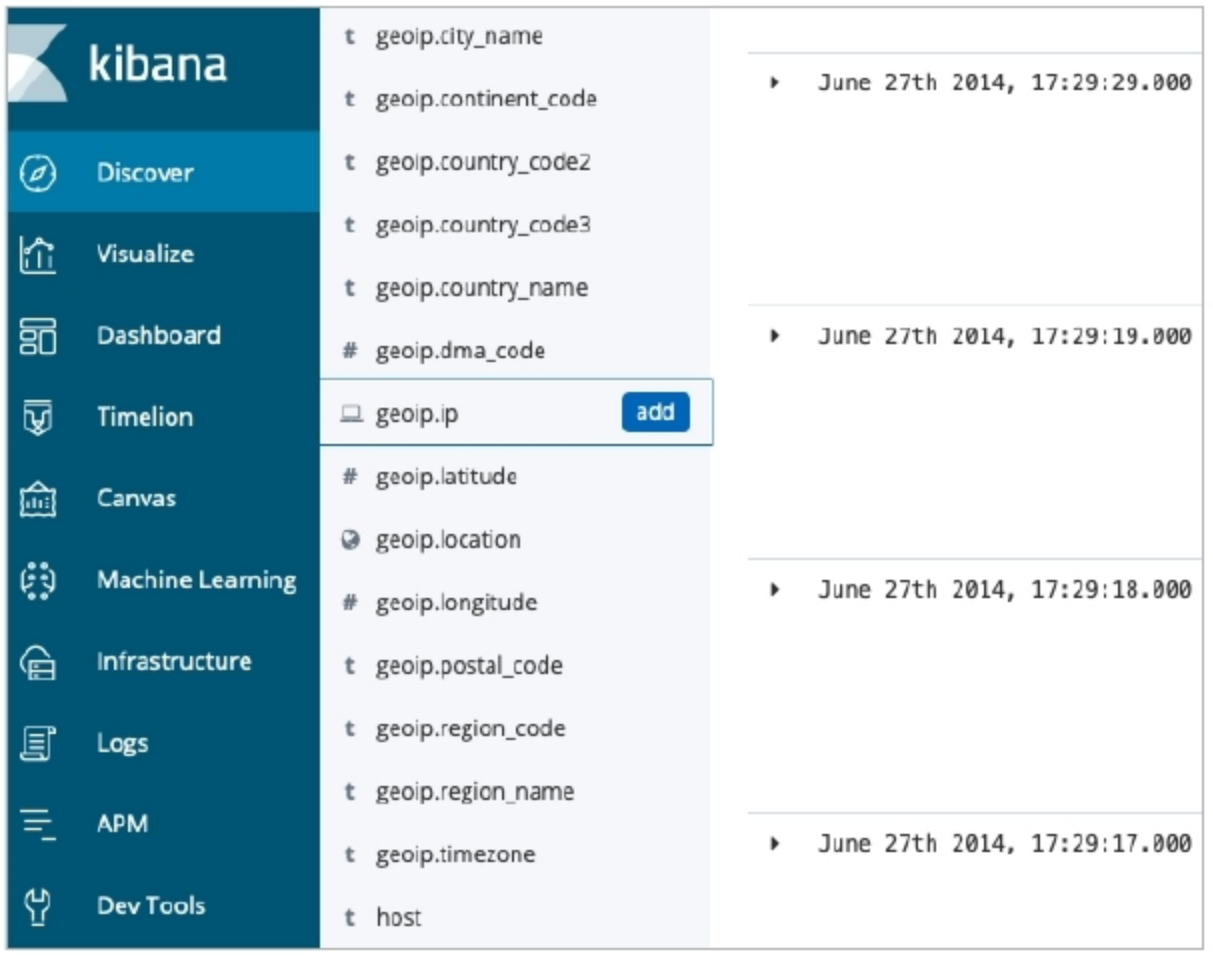

Figure 4: Data obtained after enhancing IP address using Logstash filters

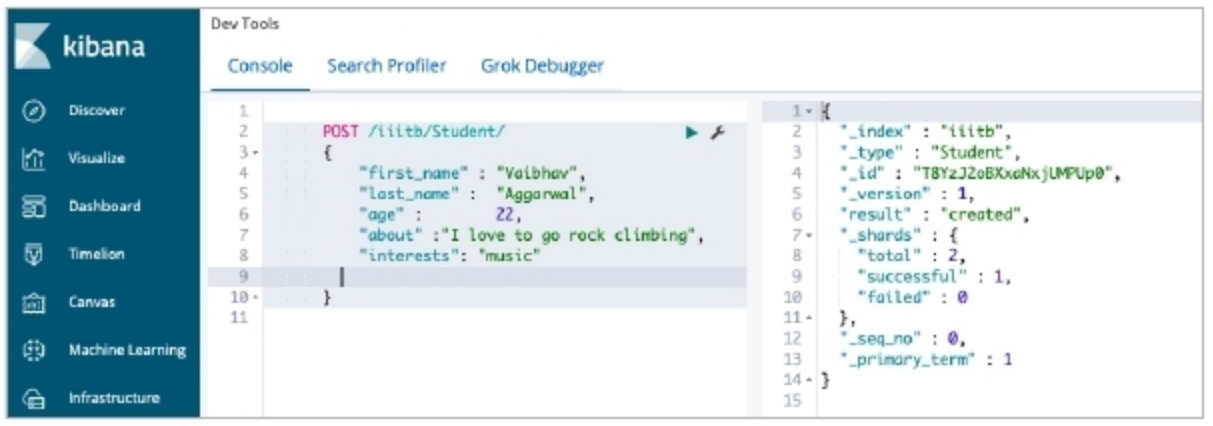

Figure 5: Dev tools in Kibana for querying Elasticsearch

when doing complex queries on data.

Indexing

Indexing is jargon for inserting data in Elasticsearch, which stores data as documents contained under an index. The index can be thought of as a table in SQL. An Elasticsearch cluster can have multiple indices, which in turn contain multiple types. These types then have multiple documents. A document is similar to a row in SQL. Documents can have multiple fields that provide information about a data object.

Here is an example:

- A POST statement is used for creating an index.
- Here, *iiitb* is an index.
- 'Student' is one of the types under *iiitb*. Others are professors, staff and security.
- Inside the brackets, we have a document under the type 'student'.

By looking at the example stated above, we observe that there wasn't a need to perform any architectural work specifically, like creating the index or defining the types of a field as in an RDBMS. Elasticsearch stores this data as documents using inverted indexing.

Inverted indexing

Elasticsearch uses an inverted index to achieve a fast full-text search over documents. An inverted index consists of all the unique words in the document, and each unique word is mapped to the list of documents containing it.

Term	DOC 1	DOC 2	DOC 3
Brown	0	1	0
Fox	0	0	1
Quick	0	1	1
Vaibhav	1	0	0
Aggar-wal	1	0	0
Music	1	0	0

Table 1

Table 1 considers three documents and six words. Each column shows if the document contains the corresponding word. The way in which Elasticsearch stores this data is called inverted indexing.

The inverted index may hold a lot more information than just the list of the documents containing a unique term. It may have a count of documents, the number of times the term appears in the document, the length of the document, and many other valuable insights.

Indexing involves tokenisation and normalisation, which is essentially an analysis of the document. The data from each field is broken into terms, and then normalised into a standard form to improve 'searchability'.

Token filters used in analysis to increase search efficiency are listed below.

1. ***Stop words:*** Removing frequently occurring words like *the*, *is*, *they*, etc.
2. ***Lower casing:*** All words are changed into lower case for better results.
3. ***Stemming:*** This involves using only the root word. The extra tense-defining part is removed; e.g., swimming and swimmer can be stemmed to swim.
4. ***Synonyms:*** This filter merges the occurrences of words with the same meaning.